# ChefNextDoor
# Home-Cooked Food Delivery Platform

Metropolitan University
Department of Software Engineering

**Course Title:** Web Programming Practice Lab
**Course Code:** SWE-322: WP
**Topic:** Technical Documentation of ChefNextDoor

**Submitted To:**

Lukman Hossain Nakib
Adjunct Faculty
Department of Software Engineering

**Submitted By:**

Name: Fabiha Sanjeda Nuva
ID: 241-134-010
Batch: 6th

**Submission date:** 30 April 2026

# ChefNextDoor

*From Their Home to Your Heart*

## Technical Documentation

Web Application Final Project

# 1. Database Schema

ChefNextDoor uses a MySQL database named chefnextdoor with 7 tables. Below is a detailed description of each table and its columns.

## 1.1 users

The main authentication table. Stores all users regardless of role.

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | INT | PK, AUTO_INCREMENT | Unique user identifier |
| name | VARCHAR(100) | NOT NULL | Full name of the user |
| email | VARCHAR(120) | UNIQUE, NOT NULL | Email address used for login |
| password | VARCHAR(255) | NOT NULL | Bcrypt hashed password |
| role | ENUM | NOT NULL | Values: customer, chef, admin |
| phone | VARCHAR(20) | NULL | Optional phone number |
| profile_image | VARCHAR(255) | NULL | Path to profile image |
| created_at | TIMESTAMP | DEFAULT NOW() | Account creation timestamp |

## 1.2 chef_profiles

Stores additional information for chef users. Linked to the users table.

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | INT | PK, AUTO_INCREMENT | Unique profile identifier |
| user_id | INT | FK -> users.id | References the chef in users table |
| bio | TEXT | NULL | Chef's personal description |
| specialty | VARCHAR(100) | NULL | Cuisine specialty (e.g. Biryani) |
| location | VARCHAR(150) | NULL | Chef's location |
| rating | DECIMAL(2,1) | DEFAULT 0 | Cached average rating |
| is_approved | TINYINT(1) | DEFAULT 1 | Admin approval status |
| created_at | TIMESTAMP | DEFAULT NOW() | Profile creation timestamp |

## 1.3 dishes

Stores all dishes listed by chefs on the platform.

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | INT | PK, AUTO_INCREMENT | Unique dish identifier |
| chef_id | INT | FK -> users.id | References the chef who owns the dish |
| title | VARCHAR(150) | NOT NULL | Name of the dish |
| description | TEXT | NULL | Dish description |
| price | DECIMAL(10,2) | NOT NULL | Price in Bangladeshi Taka |
| image | VARCHAR(255) | NULL | Filename of uploaded dish image |
| category | VARCHAR(100) | NULL | Category (e.g. Rice, Curry, Dessert) |
| availability | TINYINT(1) | DEFAULT 1 | 1=available, 0=unavailable |
| stock | INT | DEFAULT 0 | Number of portions available |
| created_at | TIMESTAMP | DEFAULT NOW() | Dish creation timestamp |

## 1.4 orders

The main order table. Records every order placed by a customer.

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | INT | PK, AUTO_INCREMENT | Unique order identifier |
| customer_id | INT | FK -> users.id | The customer who placed the order |
| chef_id | INT | FK -> users.id | The chef receiving the order |
| total_price | DECIMAL(10,2) | NULL | Total order price in Taka |
| delivery_address | TEXT | NULL | Customer's delivery address |
| status | ENUM | DEFAULT 'pending' | pending, accepted, preparing, out_for_delivery, delivered, cancelled |
| payment_status | ENUM | DEFAULT 'unpaid' | unpaid or paid |
| created_at | TIMESTAMP | DEFAULT NOW() | Order placement timestamp |

## 1.5 order_items

Stores individual dishes within each order.

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | INT | PK, AUTO_INCREMENT | Unique item identifier |
| order_id | INT | FK -> orders.id | References the parent order |
| dish_id | INT | FK -> dishes.id | References the ordered dish |
| quantity | INT | DEFAULT 1 | Number of portions ordered |
| price | DECIMAL(10,2) | NULL | Price per unit at time of order |
| subtotal | DECIMAL(10,2) | NULL | quantity x price |

## 1.6 reviews

Stores customer reviews and ratings for chefs after delivered orders.

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | INT | PK, AUTO_INCREMENT | Unique review identifier |
| customer_id | INT | FK -> users.id | The customer writing the review |
| chef_id | INT | FK -> users.id | The chef being reviewed |
| order_id | INT | FK -> orders.id | The order being reviewed |
| rating | INT | CHECK 1-5 | Star rating from 1 to 5 |
| comment | TEXT | NULL | Optional written comment |
| created_at | TIMESTAMP | DEFAULT NOW() | Review submission timestamp |

## 1.7 favorites

Stores dishes that customers have saved as favourites.

| Column | Type | Constraints | Description |
|---|---|---|---|
| id | INT | PK, AUTO_INCREMENT | Unique favourite identifier |
| customer_id | INT | FK -> users.id | The customer who saved the dish |
| dish_id | INT | FK -> dishes.id | The saved dish |
| created_at | TIMESTAMP | DEFAULT NOW() | Date the dish was saved |

## 2. Entity Relationship Diagram (ERD)

The ERD below describes how the 7 tables in ChefNextDoor relate to each other.

**Table Relationships:**

- users (1) --> (many) chef_profiles: One user can have one chef profile
- users (1) --> (many) dishes: One chef can list many dishes
- users (1) --> (many) orders: One customer can place many orders
- orders (1) --> (many) order_items: One order can contain many dishes
- dishes (1) --> (many) order_items: One dish can appear in many orders
- users (1) --> (many) reviews: One customer can write many reviews
- users (1) --> (many) favorites: One customer can save many dishes
- dishes (1) --> (many) favorites: One dish can be saved by many customers

*Note: Please refer to the dbdiagram.io ERD diagram submitted alongside this document for the full visual diagram.*

# 3. Setup Instructions

Follow these steps to run ChefNextDoor on your local machine.

## 3.1 Requirements

- XAMPP (includes Apache, PHP 8.x, MySQL)
- Composer (PHP package manager)
- A web browser (Chrome, Firefox, Safari)
- Git (optional, for cloning)

## 3.2 Installation Steps

1. Download or clone the project from GitHub: https://github.com/fabihanuva/ChefNextDoor
2. Place the ChefNextDoor folder inside your XAMPP htdocs directory: C:\xampp\htdocs\ (Windows) or /Applications/XAMPP/xamppfiles/htdocs/ (Mac)
3. Open Terminal or Command Prompt, navigate to the project folder and run: composer install
4. Copy the .env.example file and rename it to .env
5. Open phpMyAdmin at http://localhost/phpmyadmin
6. Create a new database named: chefnextdoor
7. Click on the chefnextdoor database, then click the SQL tab
8. Copy the contents of sql/schema.sql and paste it into the SQL tab, then click Go
9. Open the .env file and update the database credentials:

```
DB_HOST=localhost
DB_NAME=chefnextdoor
DB_USER=root
DB_PASS=
BASE_PATH=/ChefNextDoor/public
```

10. Start Apache and MySQL from the XAMPP Control Panel
11. Open your browser and visit: http://localhost/ChefNextDoor/public

## 3.3 Test Accounts

You can register new accounts directly from the app. Select your role (Chef or Customer) during registration.

## 3.4 Image Uploads

Dish images are stored in the uploads/dishes/ folder. Make sure this folder exists and has write permissions:

```
chmod -R 777 uploads/
```

## 3.5 Troubleshooting

- 404 errors: Make sure mod_rewrite is enabled in Apache and AllowOverride is set to All in httpd.conf
- Database errors: Verify your .env credentials match your MySQL setup
- Image not showing: Check that the uploads/dishes folder exists and is writable